BRNO UNIVERSITY OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY

ERASMUS+ GUEST LECTURE

HELLENIC MEDITERRANEAN UNIVERSITY · HERAKLION · 2026

Knowledge Graphs

Teaching the web what things mean – and letting an AI ask the questions

Radek Burget
Faculty of Information Technology, Brno University of Technology

burgetr@fit.vut.cz
Czech Republic

One Question, Three Sources

YOU ARE BUILDING AN APP FOR ERASMUS STUDENTS

"Show me archaeological sites within 30 km of my university that are open on Sunday."

What the app has to combine

- Where is **my university**? → a university registry
- Which places are **archaeological sites**, and where? → e.g. Wikidata
- When are they **open**? → e.g. OpenStreetMap
- No single document contains the answer

Why not just…

- **search the web?** You get pages about Knossos – reading and combining them is still your job
- **ask an LLM?** A fluent answer – but no sources; sites, distances or opening hours may be **outdated or invented**

> Applications need facts they can **combine** and **check**: things with global names, relations with explicit meaning.

Web of Documents vs. Web of Data

World Wide Web

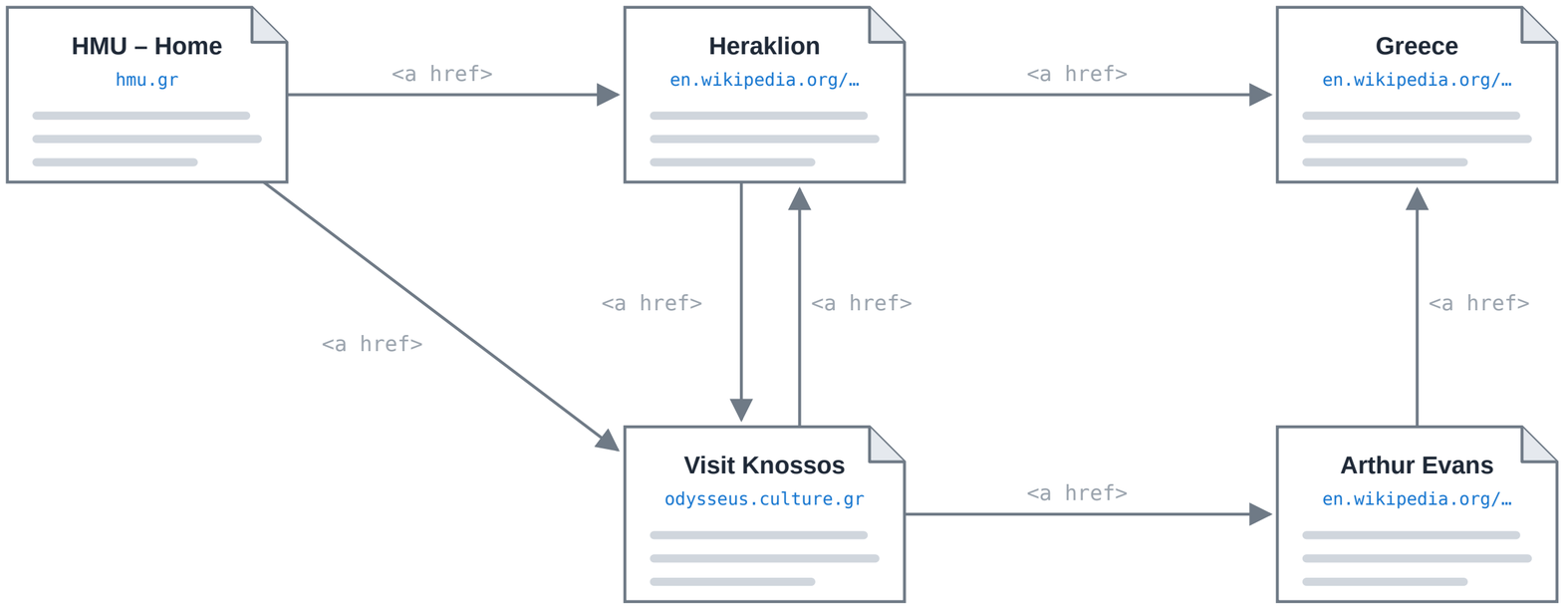
- The unit is a **document**
- Links go from page to page
- Meaning is implicit – in the text, **for humans**
- HTTP, URL, HTML

Web of Data

- The unit is a **statement about a thing**
- Links go from thing to thing
- Meaning is explicit – **for machines**
- HTTP, IRI, **RDF**, **SPARQL**

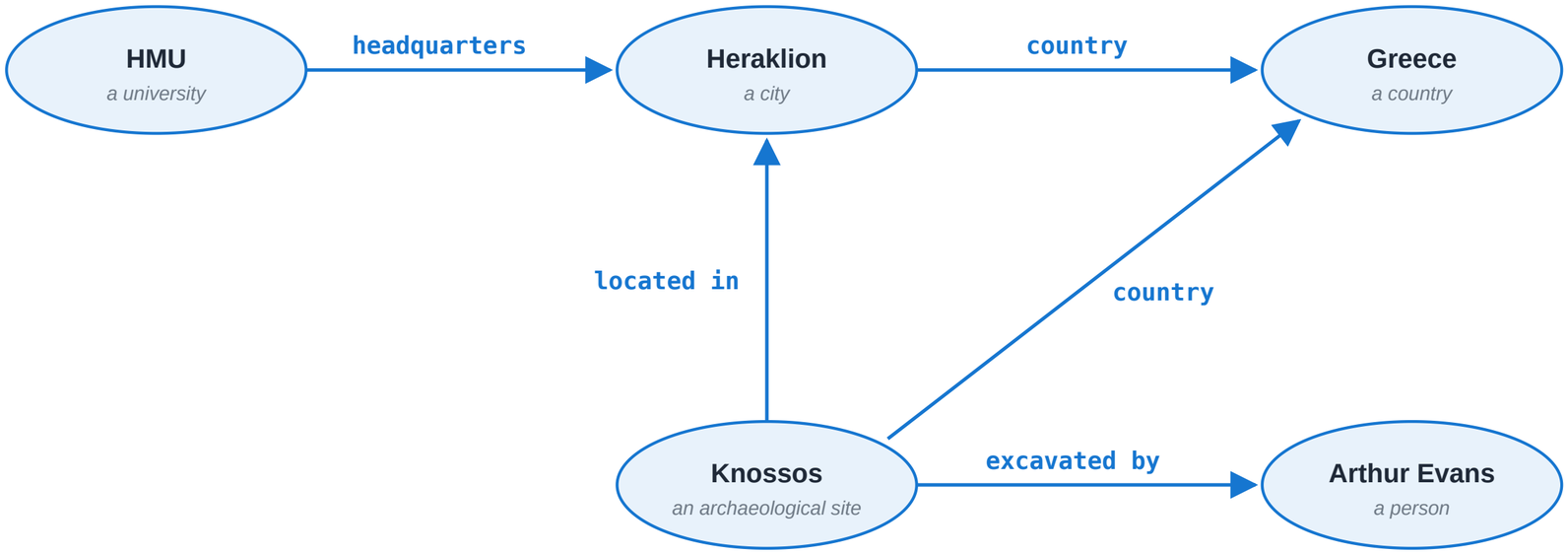
Berners-Lee, T., Hendler, J., Lassila, O.: **The Semantic Web.** Scientific American 284(5), 34–43 (2001)

Web of Documents: Pages and Links



- Nodes are **pages**, edges are **hyperlinks** – and every link says just “see this”
- **Why** does the HMU page link to Heraklion? Only a human reading the text knows.

Web of Data: Things and Predicates



- Nodes are the **things themselves** – not pages about them

25 Years Later

Promised in 2001

- Autonomous software agents negotiating your appointments
- Logic, proofs, a “web of trust”

What we actually got

- **RDF**, **SPARQL**, **JSON-LD**, shared vocabularies
- Renamed: **knowledge graphs** (Google, 2012)
- Wikidata: **100+ million things**, billions of facts, free to query
- The agents? They came as **LLMs** – we'll meet one today

PART 01

Knowledge Graphs in Ten Minutes

Things, not strings: how to write down facts so that a machine can use them

Machine-Readable != Machine-Understandable

Three rental websites export their offers as XML:

Site A	Site B	Site C
`<size>2</size>`	`<size>65</size>`	`<size>65 m2</size>`
number of bedrooms	floor area, a number	floor area with a unit

- The **meaning** of `<size>` lives in somebody's code, not in the data
- Every integration = reading docs + writing a converter
- And what is `<location>Heraklion</location>`? The city? The municipality? The regional unit? A hotel named **Heraklion**?

> We need two things: **global names for things** and **shared meaning of properties**.

Step 1: Give Everything a Global Name

- Every thing gets an **IRI** – a URL-like identifier (generalised URL, Unicode allowed)
  - Heraklion: `http://www.wikidata.org/entity/Q160544`
    - or `http://dbpedia.org/resource/Heraklion`
  - Greece: `http://www.wikidata.org/entity/Q41`
    - or `http://dbpedia.org/resource/Greece`
- Not just pages: **cities, people, books, properties, concepts**
- Prefixes keep it readable: `wd:Q160544` = `http://www.wikidata.org/entity/Q160544`

Where do IRIs come from?

- **My data** → my own domain, e.g. `https://fit.vut.cz/student/938272`
- **Well-known things** → reuse public knowledge bases (Wikidata, DBpedia)

Linked Data rule of thumb

- Use **HTTP** IRIs
- When someone opens one, **return useful data**
- **Link to other people's IRIs**

Short IRIs: Prefixes

NOBODY WANTS TO TYPE `http://www.wikidata.org/entity/` A HUNDRED TIMES

Namespace + local name

`http://www.wikidata.org/entity/`**`Q160544`**

↓

`wd:`**`Q160544`**

- Declare the prefix **once** at the top of the file:
  - Turtle: `@prefix wd: <http://www.wikidata.org/entity/> .`
  - SPARQL: `PREFIX wd: <http://www.wikidata.org/entity/>`
- Just **shorthand**: the data always contains the full IRI
- The name `wd:` is a convention – pick any, look up common ones at prefix.cc

prefix	namespace	used for
`wd:`	`http://www.wikidata.org/entity/`	Wikidata items
`wdt:`	`http://www.wikidata.org/prop/direct/`	Wikidata properties
`rdf:`	`http://www.w3.org/1999/02/22-rdf-syntax-ns#`	RDF core (`rdf:type`)
`schema:`	`https://schema.org/`	schema.org
`dbr:`	`http://dbpedia.org/resource/`	DBpedia

Step 2: Statements as Triples



SUBJECT
what I talk about

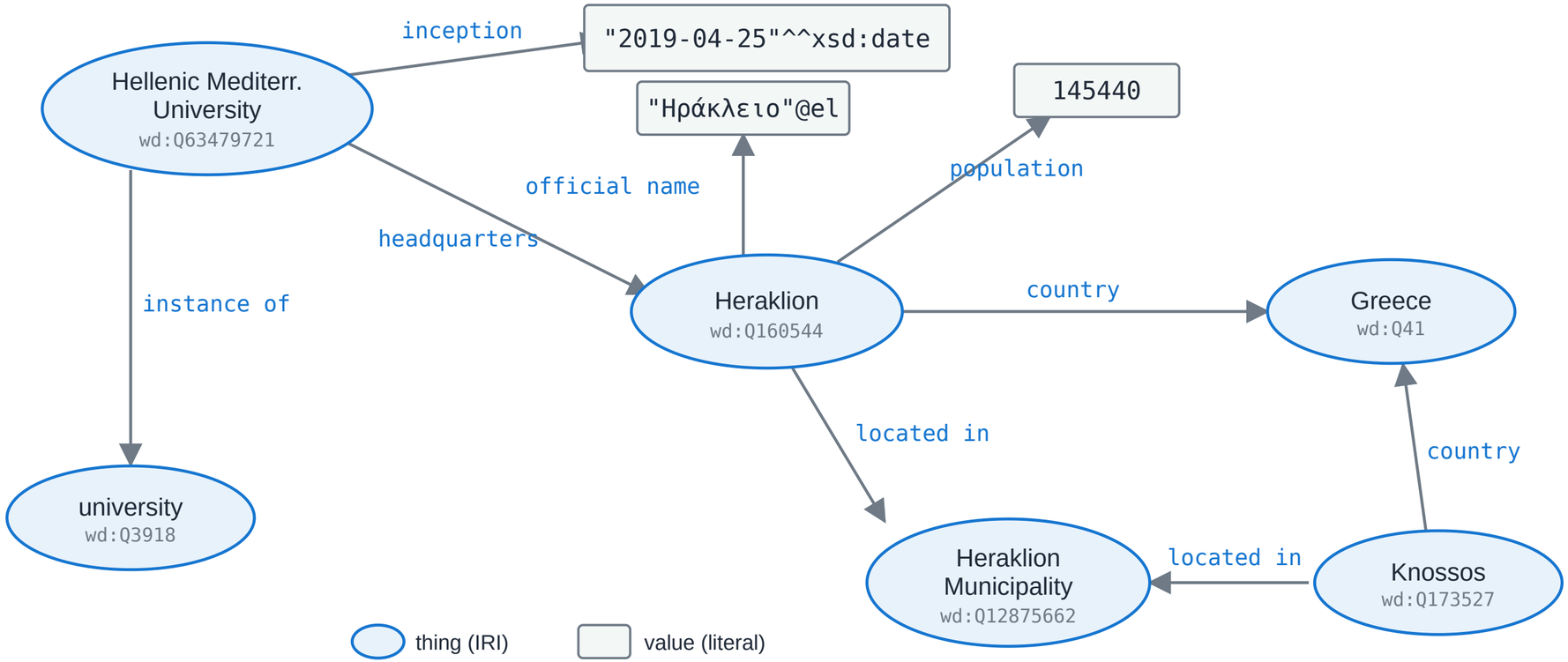
PREDICATE
what I claim about it

OBJECT
another thing, or a value

“Heraklion is in Greece.”

- **RDF** (Resource Description Framework): every fact is a **subject – predicate – object** triple
- Subject and predicate are always IRIs; the object is an IRI or a **literal** (string, number, date)
- Anybody can publish triples about anything

Triples Make a Graph



- Triples sharing an IRI **connect automatically** – no foreign keys, no schema migration
- Literals can carry a **language** (`@el`) or a **datatype** (`^^xsd:date`)

Writing It Down: Turtle

THE SAME GRAPH WRITTEN IN TEXT

Turtle Syntax

- One statement = `subject predicate object .`
- `;` repeats the subject, `,` repeats subject and predicate
- `#` starts a comment – Wikidata IDs are not meant to be read by humans

Adding my own facts? One more prefix:

```
@prefix schema: <https://schema.org/> .
@prefix ex:     <https://example.org/my-erasmus/> .

ex:me  a  schema:Person ;        # a = rdf:type
       ex:visited  wd:Q160544 , wd:Q173527 .
```

- Three vocabularies, three owners – **nobody had to agree in advance**
- Same graph can be written as **JSON-LD** (we'll see it in web pages) or N-Triples

Vocabularies (a.k.a. Ontologies)

Reuse first

- **schema.org** – people, places, products, events, recipes… the vocabulary of the web
- **Wikidata properties** – `wdt:P17` country, `wdt:P1082` population, … 12 000+
- **Dublin Core** (documents), **FOAF** (people), **GeoSPARQL** (geometry), **SKOS** (taxonomies)

…define your own if you must

- An ontology is just **more RDF**: classes, properties, how they relate
- Languages for it: RDFS, OWL (we skip the details today)
- We will see a **custom ontology** at the end – for describing rendered web pages
- Search: Linked Open Vocabularies

PART 02 · LIVE

Asking the Graph: SPARQL

Pattern matching over billions of facts – everything on the following slides runs live

SPARQL: Draw the Pattern, Get the Matches

WHICH GREEK ISLANDS HAVE THE MOST INHABITANTS?

```
SELECT ?island ?islandLabel ?population WHERE {
  ?island wdt:P31  wd:Q23442 ;      # ?island is an island
          wdt:P17  wd:Q41 ;         # … in Greece
          wdt:P1082 ?population .   # … with a population
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
}
ORDER BY DESC(?population)
LIMIT 10
```

- The query is a **graph with holes** (`?variables`); the engine finds every way to fill them
- No tables, no JOIN: a join is just **the same variable used twice**
- ▶ Run on query.wikidata.org

Wait – Why Is Rhodes There Twice?

islandLabel	population
Crete	623 065
Euboea	191 206
Rhodes	**127 613**
Rhodes	**115 490**
Corfu	**100 854**
Corfu	**99 134**
Lesbos	83 800

- Wikidata stores the population **for 2011 and for 2020** – both are true!
- Multi-valued property ⇒ every combination becomes a row
- Your “top 10” silently became a top 8

> Fixes: aggregate (`MAX` , `GROUP BY`), filter by the date qualifier, or ask only for the **preferred** value.

A knowledge graph stores **facts with history**, not a single “current row”.

Results Don't Have to Be Tables

ALL ARCHAEOLOGICAL SITES IN THE CRETE REGION – ON A MAP

```
#defaultView:Map
SELECT DISTINCT ?site ?siteLabel ?coords ?image WHERE {
  ?site wdt:P31/wdt:P279* wd:Q839954 ;   # an archaeological site (or any subtype)
        wdt:P131+ wd:Q1267522 ;          # somewhere inside the Crete Region
        wdt:P625 ?coords .               # with coordinates
  OPTIONAL { ?site wdt:P18 ?image }
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
}
```

- `/`, `*`, `+` are **property paths**: “type or a subtype of it”, “located in, transitively”
- **~250 sites** – click a dot to see the photo
- ▶ Run on query.wikidata.org

… or a Photo Album

MONASTERIES OF CRETE

```
#defaultView:ImageGrid
SELECT DISTINCT ?m ?mLabel ?image WHERE {
  ?m wdt:P31 wd:Q44613 ;           # a monastery
     wdt:P131+ wd:Q1267522 ;       # in the Crete Region
     wdt:P18 ?image .              # that has a photo
  SERVICE wikibase:label { bd:serviceParam wikibase:language "en". }
}
```

- Only change: one comment line selects the view
- ▶ Run on query.wikidata.org

One City, Eighty Names

DBPEDIA: “WHICH OTHER IRIS MEAN THE SAME HERAKLION?”

```
SELECT ?same WHERE { dbr:Heraklion owl:sameAs ?same }
```

```
wd:Q160544                    ← Wikidata
sws.geonames.org/261745       ← GeoNames
viaf.org/viaf/162900004       ← library catalogue
musicbrainz.org/area/4f3c…    ← music database
it.dbpedia.org/Candia         ← Italian Wikipedia
tr.dbpedia.org/Kandiye        ← Turkish Wikipedia
el.dbpedia.org/Ηράκλειο_Κρήτης
… 80+ links
```

- `owl:sameAs` = “these two IRIs denote **the same thing**”
- Entity matching that **somebody already did and published**
- Spot the history: **Candia** (Venetian), **Kandiye** (Ottoman)
- But beware: the `sameAs` claim can't be always trusted

▶ Run on dbpedia.org/sparql

OpenStreetMap as a Graph

```
PREFIX osmkey: <https://www.openstreetmap.org/wiki/Key:>
PREFIX osmrel: <https://www.openstreetmap.org/relation/>
PREFIX ogc:    <http://www.opengis.net/rdf#>

SELECT ?amenity (COUNT(?x) AS ?count) WHERE {
  osmrel:453129 ogc:sfContains ?x .     # everything inside the island of Crete
  ?x osmkey:amenity ?amenity .
  FILTER(?amenity IN ("restaurant", "cafe", "bar", "place_of_worship"))
}
GROUP BY ?amenity
ORDER BY DESC(?count)
```

amenity	count
place_of_worship	**3 824**
restaurant	2 496
cafe	976

- Crete has **more churches than restaurants** (at least on the map)
- Plus **500 mapped beaches** – try `osmkey:natural "beach"` ▶
- ▶ Run on qlever.dev

PART 03

Knowledge Graphs Hidden in Web Pages

Where search engines get their facts from

Look at the Page Source

A (made-up) taverna page in Heraklion contains this block, invisible to visitors:

```
<script type="application/ld+json">
{ "@context": "https://schema.org",
  "@type": "Restaurant",
  "name": "Taverna O Minotavros",
  "servesCuisine": "Cretan",
  "address": {
    "@type": "PostalAddress",
    "addressLocality": "Heraklion" },
  "aggregateRating": {
    "@type": "AggregateRating",
    "ratingValue": "4.7",
    "reviewCount": "312" }
}
</script>
```

- **JSON-LD**: looks like plain JSON, **is** RDF
- `@context` maps short keys to IRIs: `"name"` ⇒ `https://schema.org/name`
- Vocabulary: **schema.org** (Google, Microsoft, Yahoo, Yandex)
- Older alternatives: Microdata, RDFa (attributes inside the HTML)

> ~70 % of websites that annotate anything use JSON-LD; the average annotated page now carries **~57 triples**.

Why Would Anyone Bother?

- Search engines turn annotations into **rich results**: stars, prices, opening hours, recipes, events
- The website gets **more clicks** – a purely selfish reason, and that's why it works

Well covered

- Shops and products
- Recipes, events, local businesses
- News articles

The long tail

- Timetables, public records, small shops, university pages…
- **No annotations at all**
- And annotations may be **marketing**, not truth

BONUS · OUR RESEARCH

Chatting with a Web Page

Turning any rendered page into a knowledge graph – and letting an AI write the queries

FitLayout: A Web Page as a Graph

FitLayout (our open-source framework) renders a page in a headless browser and describes **what it looks like** in RDF:

- **Boxes** – every rendered element with position, size, font, colour
- **Visual areas** – blocks a human would perceive (name, phone, menu…)
- **Spatial relations** – `above`, `below`, `leftOf`, `rightOf`
- **Tags** – labels such as **e-mail**, **phone**, **room** (for machine learning)



> Example: a staff page at FIT BUT – no JSON-LD, yet **"E-mail"** is **left of** the address, the graph says so.

Our Own Vocabulary (Ontology)

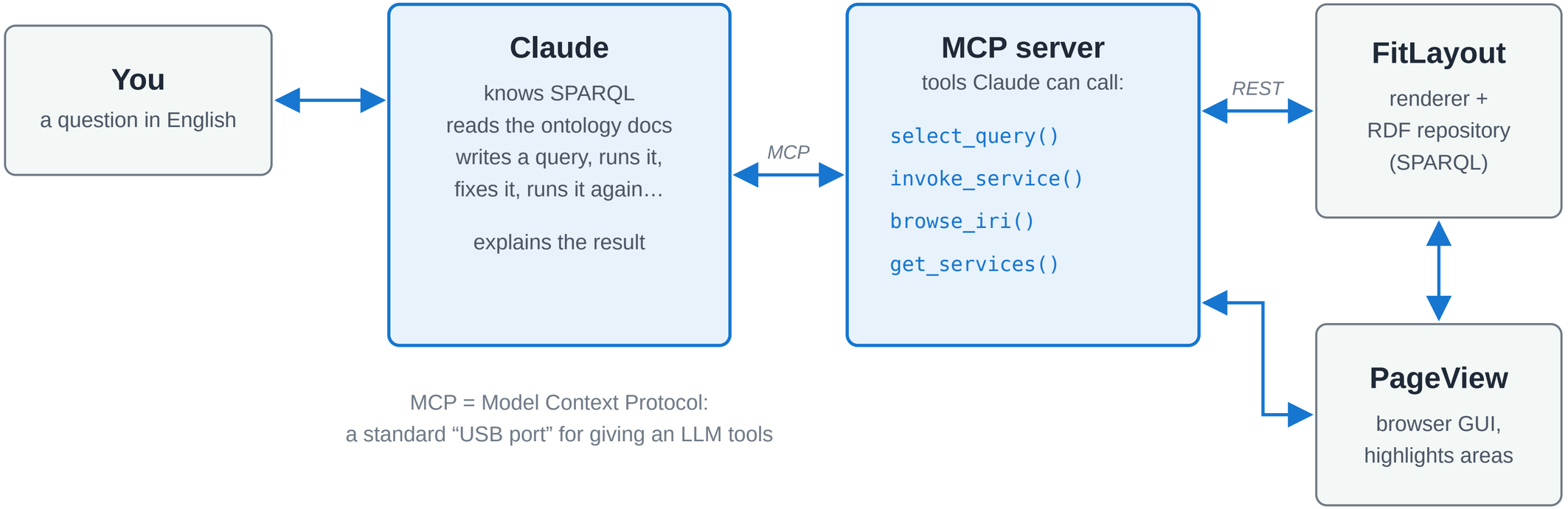
No existing vocabulary describes **rendered boxes and their positions** – so we defined one (`box:`, `segm:`).

```
r:art1450#a124  a  segm:Area ;                # the e-mail address on the page
    segm:text          "burgetr@fit.vut.cz" ;
    box:fontSize       14.0 ;
    box:underline      1.0 ;
    box:bounds         r:art1450#a124-rect-b ;
    segm:belongsTo     r:art1450 ;               # the area tree of the page
    segm:isChildOf     r:art1450#a121 ;          # parent area
    segm:containsBox   r:art22#b488 ;            # the rendered <td>
    segm:hasTag        r:tag-person--email ;     # label for machine learning
    segm:isInRelation r:rd-art1450-a124-rightOf-a122 .
```

- It is **still RDF**: the same Turtle, the same SPARQL, the same tools as for Wikidata

PageView: The Same Area in the GUI

Let Claude Write the SPARQL



MCP = Model Context Protocol:
a standard “USB port” for giving an LLM tools

- Claude gets the **ontology documentation** and a tool to **run SPARQL**
- It writes a query, looks at the result, **corrects itself** if needed

Live Demo

ASK A STAFF PAGE ANYTHING

Suggested questions

1. “What is this page about?”
2. “What is the largest text on the page?”
3. “What is the e-mail, phone and room of this person?”
4. “Highlight the phone number in the browser.”
5. “Render the page of another colleague and find their room.”

Watch for

- The **SPARQL** Claude generates
- How it **uses the ontology** (areas, relations, tags)
- When it **fixes** its own query

Backup: “What Is the Largest Text?”

Claude's query

```
PREFIX box:  <http://fitlayout.github.io/ontology/render.
PREFIX segm: <http://fitlayout.github.io/ontology/segment
PREFIX r:    <http://fitlayout.github.io/resource/>

SELECT DISTINCT ?text ?size WHERE {
  ?area segm:belongsTo r:art1450 ;
        box:fontSize   ?size ;
        segm:text      ?text .
  FILTER(STRLEN(?text) > 0)
}
ORDER BY DESC(?size)
LIMIT 5
```

Result

text	size
BURGET	30.0
RADEK	30.0
RADEK BURGET	30.0
CONTACT	28.0
Department of Information Systems	24.0

- The largest text is the **person's name** – split into two leaf areas plus their parent
- Then the section heading and the department

Backup: “E-mail, Phone and Room?”

Claude's query – follows the spatial relations

```
PREFIX segm: <http://fitlayout.github.io/ontology/segment
PREFIX r:    <http://fitlayout.github.io/resource/>

SELECT ?label ?value WHERE {
  ?l   segm:belongsTo r:art1450 ;
       segm:text ?label ;
       segm:isInRelation ?rel .
  ?rel segm:hasRelationType r:rel-leftOf ;
       segm:hasRelatedRect  ?v .
  ?v   segm:text ?value .
  FILTER(?label IN ("E-mail", "Work Phone", "Room"))
}
```

Result

label	value
E-mail	burgetr@fit.vut.cz
Work Phone	+420 54114 1320
Room	C215 Office

- No pixel arithmetic: the **graph already knows** that the label is **left of** the value
- Claude found `r:rel-leftOf` in the ontology docs, not in its training data

Why Does This Work?

Claude is good at

- SPARQL syntax and idioms (aggregates, paths, `OPTIONAL`, `FILTER`)
- Reading a **schema description** and following it
- **Trial and error**: run, inspect, refine

…as long as it can check

- Without a tool, an LLM asked for a Wikidata query may **invent** a plausible-looking `wdt:P…` or `wd:Q…`
- With an endpoint, wrong IDs show up as **empty results** and get fixed

Knowledge graph = **explicit, checkable facts**. LLM = **natural-language interface**. Together they cover each other's weaknesses.

Where It Hurts

- **Timeouts and rate limits** – a public endpoint won't finish your big query
  - Wikidata had to be **split in two graphs** in 2025 – scholarly articles had outgrown one server
- **Uneven coverage** – Wikidata knows everything about Knossos and nothing about the taverna next door
- **Quality** – historical values, sloppy `owl:sameAs`, marketing data in JSON-LD
- **Availability** – someone else's server: up today, `502` tomorrow
- Remedy: **download the dump**, load it into a local triple store (QLever, Oxigraph, Fuseki)

Take-Aways

1. Things, not strings

Give things **global IRIs**, write facts as **triples**; graphs merge for free.

2. Reuse vocabularies

schema.org, Wikidata properties… define your own **only** when nothing fits.

3. SPARQL + LLM

Public graphs are one query away – and an AI with a SPARQL tool **writes those queries well**.

> Try it tonight: query.wikidata.org → **Examples** – hundreds of ready-made queries to tweak.

ΕΥΧΑΡΙΣΤΏ!

Questions?

Radek Burget · burgetr@fit.vut.cz · FitLayout: github.com/FitLayout